

Business logic, page by page

For each page of the cockpit: the endpoint it calls, its parameters, the chain of functions it runs through, and the maths behind the calculation. Every section ends with the answer to give out loud.

Engine figures, seed 42, 100,000 simulated years. Every value quoted here comes from deck/figures.json, written by the pipeline.

Overview

Page	Endpoint	Parameters	Cost
Overview	the four below	—	all cached
Telemetry	GET /api/telemetry/summary/ GET /api/frequency/	none	cached
Frequency	GET /api/frequency/ GET /api/assets/	threshold, window	~1 s
Severity	GET /api/severity/	none	cached
Simulation	POST /api/simulate/	7 (see §4)	a few s

- **One architectural rule:** the view parses the request, calls the engine, serializes. No risk maths in Django, none in the frontend.
- **Two levels of cache:** `get_dataset()` loads and fits once for all — the CSVs do not change — while `get_frequency()` and `get_simulation()` are memoized per parameter set.
- **Every result object exposes** `to_explanation()` — the numbered trace the CLI prints, the API serves and the UI displays. One version of each figure.

1. Telemetry — what the sensors saw

Endpoint and parameters

```
GET /api/telemetry/summary/      (no parameters)
GET /api/frequency/
```

- The second call serves only the **last stage of the funnel**: the incident rate the simulation consumes.
- No parameters: normalization depends on no convention.

Call chain

1. `load_assets()` — reads the reference for the 20 assets.
2. `load_siem()` / `load_edr()` — read each export and translate it into canonical `SecurityEvents`. This is where the two severity scales are brought into one shared vocabulary.
3. `severity_from_siem_label()` — Low/Medium/High/Critical → class.
4. `severity_from_edr_risk()` — score 0–999 → class, by cut points.
5. `merge_feeds()` — **deduplicates** over the union of both feeds and produces the `NormalizationReport`.
6. `observed_window()` — derives the observation window from the events themselves.
7. `summarize_telemetry()` — aggregates into weekly buckets, severity mix, top techniques.

The maths

- **Severity normalization.** The SIEM gives a label, the EDR a score. EDR cut points: < 50 low, 50–69 medium, 70–93 high, ≥ 94 critical. Numeric score attached to each class: 0.25 / 0.5 / 0.75 / 1.0.
- **Deduplication key:** the triple (`asset_id`, `technique`, `observed_at`). Two reports carrying the same triple describe the same real event.
- **Deduplicated over the union, not by a join.** Each feed already repeats keys internally — 761 times in the SIEM, 543 in the EDR — so an inner join would multiply those duplicates and return 13,055 rows instead of the 12,343 true matches.
- **Merge rule: the worst severity wins.** If the EDR sees critical and the SIEM medium, critical is kept: the tool closest to the endpoint saw something. Averaging would dilute a strong signal into a weak one.
- **Annualization factor** = 365 / observed days = 365 / 212 = **1.7217** (1 Nov 2025 → 31 May 2026). Recomputed from the data, never hardcoded.

Figures produced

Quantity	Value	Where it comes from
Raw rows	45,840	sum of both exports
Distinct events	32,193	after deduplication
Duplicates absorbed	13,647	12,343 of them across feeds
Inflation avoided	42.4%	what a naive concatenation would have added

SAY THIS *The two tools are not complementary, they are partly redundant: 12,343 events carry the same asset, the same technique and the same timestamp on both sides. I deduplicate on that triple keeping the worst severity, which avoids 42.4% of inflation on every downstream rate.*

2. Frequency — how often we are attacked

Endpoint and parameters

GET /api/frequency/?severity_threshold=high&session_window_hours=24

GET /api/assets/?severity_threshold=high&session_window_hours=24

Parameter	Default	Effect
severity_threshold	high	minimum severity to count as an attack
session_window_hours	24	silence that opens a new episode

These two parameters are conventions, not measurements. They are exposed as dials in the UI rather than hidden in the code.

Call chain

1. `get_frequency(threshold, window)` — memoized per parameter pair.
2. `estimate_frequency()` — orchestrates the four steps below.
3. `is_attack_grade(event, threshold)` — filter: keeps only high and above.
4. `sessionize(events, params)` — groups events into **episodes**.
5. `attack_type_for(technique)` — classifies each MITRE technique into an attack type through an explicit dictionary; the episode inherits the type of its worst event.
6. `peer_weighted_base_rate(incidents, params)` — incidents per organisation-year among peers.
7. `calibrate(lambda_detected, base_rate)` — fits `p_materialize` to reconcile the two units.

The maths

- **Episode rule.** Grouped by **asset alone** — an intruder trips whatever detections it meets, and counting each type separately would recount the same intrusion under several names. The run is cut as soon as a silence exceeds the window, measured against the **previous** event and not the first, otherwise a three-day intrusion would be split artificially.
- **λ detected** = episodes $\times$ 365 / observed days = $911 \times 1.7217 = 1,568$ / yr.
- **Peer base rate** = weighted incidents $\div$ weighted organisation-years. Numerator *and* denominator carry the same similarity kernel, one record per `company_id` — that is **1,310 organisations**.
- **The change of unit, the key point.** The telemetry counts *detected* attacks, the external base counts *loss-causing* incidents. Those are two different units.
- **Calibration:** $\lambda_{\text{incident}} = \lambda_{\text{detected}} \times p$, with p set so that $\lambda_{\text{incident}}$ equals the peer rate. Here $p \approx 1$ detection in 5,138, so $\lambda_{\text{incident}} = 0.31$ / yr.
- **The mix comes from the telemetry, the level from the base.** Each type gets $\lambda_{\text{incident}} \times$ (its episodes / total episodes).

Figures produced

Quantity	Value	Calculation
Attack-grade events	10,126	severity $\geq$ high
Episodes	911	compression 11.1×
λ detected	1,568 / yr	episodes $\times$ 365/212
λ incident	0.31 / yr	λ detected $\div$ 5,138

SAY THIS *An alert is not an attack: an intrusion produces a burst of detections. I group by asset with a 24-hour quiet window, which takes 10,126 attack-grade events down to 911 episodes, or 1,568 detected attacks a year. But the telemetry counts detections and the external base counts loss-causing incidents — two different units. I reconcile them with a calibration: one detection in 5,138 ends in a loss, giving 0.31 incident a year anchored on 1,310 comparable organisations.*

3. Severity — what one incident costs

Endpoint and parameters

GET /api/severity/ (no parameters)

No parameters: the severity model depends only on the target profile, which is fixed for this case (Retail, ETI, maturity 55).

Call chain

1. `load_incidents()` — reads and **cleans** the external base; also returns the `CleaningReport`.
2. `repair_mojibake()` — repairs the double UTF-8 encoding of the sector labels, which was splitting one sector into two.
3. `fit_severity_model()` — orchestrates the steps below, once for the pool then once per attack type.
4. `peer_weights(incidents, params)` — one weight per incident, via `sector_weight()`, `size_weight()` and `maturity_weight()`.
5. `effective_sample_size(weights)` — Kish n_{eff} , what the weighting costs.
6. `fit_lognormal(losses, weights)` — weighted maximum likelihood on the log scale.
7. `diagnose()` — assembles the evidence *against* the fit: `weighted_ks()`, `qq_points()`, `fit_pareto_tail()`, `distribution_plot()`.

The maths

- **Cleaning.** Losses of -1 are **missing values**, never €0. Two rows affected, excluded from the fit: **1,598 usable incidents of 1,600** remain.
- **Soft weighting, never a hard filter.** A continuous weight per incident, the product of three similarity factors: sector (1.0 if Retail, else 0.4), size (1.0 if ETI, else 0.6) and a Gaussian kernel on the maturity distance, of bandwidth $h = 15$.
- **Why weight.** An exact filter on the profile leaves only **112 incidents of 1,598**, and no attack type retains a credible sample. Mathematically clean, practically unusable.
- **Weighted lognormal fit** on the log scale — it is simply a weighted mean and a weighted variance:

$$\begin{aligned}w &= w_{\text{sector}} \times w_{\text{size}} \times \exp(-d^2 / 2h^2) & d &= |\text{maturity} - 55| \\ \mu &= \sum w_i \ln x_i / \sum w_i \\ \sigma^2 &= \sum w_i (\ln x_i - \mu)^2 / \sum w_i\end{aligned}$$

- **Back to euros:** median = e^{μ} and mean = $e^{(\mu + \sigma^2/2)}$. Careful: μ and σ are **not** the mean and standard deviation of the loss, but those of its logarithm.
- **Example, supply chain:** $\mu = 12.119$, $\sigma = 2.582 \rightarrow$ median **€183k**, mean **€5.1M**. The tail multiplies the mean by **28**: $e^{(\sigma^2/2)}$.
- **It is the mean that feeds the annual loss**, not the median. A decision-maker reasoning about the typical incident is wrong by a factor of 28.
- **Kish $n_{\text{eff}} = (\sum w)^2 / \sum w^2$.** It is the number of equally weighted observations that would carry as much information. It measures the **evenness** of the weights, not their size. Below **30**, the type falls back to the pooled distribution, and the substitution is recorded.
- **Evidence published with every fit:** weighted KS (here **0.118** against an indicative threshold ≈ 0.19), a QQ plot of the log-losses, and a **Pareto tail fitted as a rival** — it wins on 5 fits of 9, so VaR 99 and TVaR 99 read as lower bounds.

SAY THIS A loss is bounded at zero, has no ceiling, and spans four orders of magnitude: that is the signature of a variable whose logarithm is roughly normal, hence lognormal. I fit by weighted maximum likelihood on the logs — concretely a weighted mean and variance — one fit per attack type, because a single lognormal is rejected by Kolmogorov-Smirnov. The weights come from a similarity kernel on sector, size and maturity: I weight rather than filter, because an exact filter would leave only 112 incidents. The price of that flexibility is a smaller effective sample, which I publish type by type, with a fallback to the pooled distribution below 30.

4. Simulation — what a year costs

Endpoint and parameters

```
POST /api/simulate/
{
  "n_years": 5000,          "seed": 42,
  "severity_threshold": "high", "session_window_hours": 24,
  "loss_cap_eur": null,      "curve_points": 160,
  "histogram_bins": 40,      "include_sensitivity": true
}
```

Parameter	Default	Effect
n_years	5,000 (API) / 100,000 (CLI)	tail resolution; capped at 200,000
seed	42	bit-for-bit reproducibility
severity_threshold	high	frequency convention, see §2
session_window_hours	24	frequency convention, see §2
loss_cap_eur	99.9th percentile ≈ €23.5M	per-incident ceiling; inf to disable
curve_points	160	resolution of the OEP/AEP curves
histogram_bins	40	histogram bins (log)
include_sensitivity	true	3×3 grid — nine more runs

Call chain

1. `get_simulation(n_years, seed, threshold, window, cap)` — memoized; **the cap is part of the cache key**, two ceilings are two answers.
2. `simulate()` — the Monte Carlo loop, vectorized over blocks of years.
3. `rng.poisson(lam=λ_type, size=block)` — the incident count of each year.
4. `rng.lognormal(mean=mu, sigma=sigma, size=total)` — the price of each incident.
5. `np.minimum(losses, cap)` — the plausibility ceiling.
6. `np.bincount(years, weights=losses)` — folds incidents back into their year.
7. `summarize(annual_losses)` — AAL, median, VaR, TVaR, P(no loss), maximum.
8. `exceedance_curve(series, kind)` — AEP (annual total) and OEP (largest single loss) curves.
9. `SimulationResult.histogram(scale='log')` — log bins, zero years counted apart.
10. `sensitivity_grid()` — replays threshold × window and reports the AAL of each cell.

The maths

- **Compound Poisson.** The annual loss is a sum of a random number of random terms:
$$S = \sum_{i=1}^N X_i \quad \text{with } N \sim \text{Poisson}(\lambda) \quad \text{and} \quad X_i \sim \text{Lognormal}(\mu, \sigma)$$
- **Compound, do not multiply.** A year is not frequency times mean cost: it is zero incidents **73.7% of years**, one sometimes, two very rarely — and the sum of those cases is not the product of the means.
- **Why simulate rather than use a formula.** The expectation is analytic (Wald's identity: $E[S] = \lambda \times E[X]$), but **the distribution function of S has no closed form**. And VaR 99 and TVaR 99 are quantiles of S. The full distribution is required.
- **What fixes the number of years:** VaR 99 rests on only 1% of the years. Over 100,000 years that is 1,000 observations; over 1,000 years, ten — unusable. The tail decides, not the mean.
- **Metrics:** $\text{VaR}_\alpha = \text{quantile of order } \alpha$; $\text{TVaR}_\alpha = E[S \mid S \geq \text{VaR}_\alpha]$. $\text{TVaR} \geq \text{VaR}$ by construction — a tested invariant.
- **AEP vs OEP.** AEP = the year's total; OEP = the year's largest single loss. OEP can never exceed AEP at equal probability, and their gap measures the share of years with several incidents.
- **Plausibility ceiling.** A lognormal has no upper bound: at sigma 2.582 and over 100,000 years it eventually draws an incident costing more than the company is worth (€3.8bn uncapped). Every loss is therefore clipped at **€23.5M**, the

99.9th percentile of the losses comparable organisations were actually observed to suffer — 0.9% of draws are affected.

- **The ceiling's effect is reported, not absorbed:** the result carries `draws_capped`, `draws_total` and `aal_uncapped`.

Figures produced

Measure	Value	Business reading
AAL	€274k	the budget line
Median year	€0	73.7% of years with no incident
VaR 95	€816k	risk appetite — 1 year in 20
TVaR 95	€4.8M	mean of the worst 5% of years
VaR 99	€6.7M	1 year in 100
TVaR 99	€15.1M	the survival question

The TVaR 95 / VaR 95 ratio is 5.9: once past the threshold of the bad year, the typical loss is almost six times that threshold. That is the definition of a heavy tail.

SAY THIS *For each simulated year I draw an incident count per attack type from a Poisson distribution, price each incident in its type's lognormal, and sum. A hundred thousand years, seed 42, reproducible to the cent. I simulate rather than apply a formula because the expectation is analytic but the quantiles are not: VaR 99 and TVaR 99 need the full distribution. And I cap each single loss at the 99.9th percentile of the losses observed among peers, because an unbounded lognormal eventually draws a €3.8 billion year for a 1,200-person company — the cap removes 37.5% of the AAL, and I show that rather than absorb it.*

Cheat sheet: five questions, five thirty-second answers

Question	Answer
Why deduplicate?	12,343 events are seen by both feeds. Concatenating would inflate every rate by 42.4%.
Why episodes?	An intrusion produces a burst of alerts. Counting alerts would measure the estate's detection verbosity, not how often it is attacked.
Why a calibration?	The telemetry counts detected attacks, the external base counts loss-causing incidents. Multiplying the two directly is a category error — it is what produced €12.5bn.
Why weight rather than filter?	An exact filter leaves only 112 incidents and no modellable type. Weighting keeps everyone, with the closest dominating.
Why Monte Carlo?	The expectation has a formula, the quantiles do not. VaR 99 and TVaR 99 require the full compound distribution.

Mathematical concepts in detail: *CONCEPTS.md* (in French). Modeling decisions and their justification: *METHODOLOGY.md*. What was not built and why: *next_steps.md*.